

Experiment-7

Design, synthesis, and analysis of combinational circuits.

Objective:

The main objectives of this lab are:

- Familiarization with synthesis flow.
- Synthesis the simulation without constraints or with constraints
- Generating optimized gate-level netlist and Standard Design Constraints.

Example: Full Adder Circuit-

- Write a Verilog code for Full-Adder circuit (Any style you can use)
- Simulate with NC launch
- Simulate without NC launch
- Synthesis using TCL command

Design information and block diagram-

A **Full Adder** is a combinational logic circuit that performs the addition of three binary digits: two input bits (A, B) and a carry-in bit (Cin). It produces two outputs:

- **Sum (S):** Represents the least significant bit (LSB) of the addition.
- **Carry-out (Cout):** Represents the most significant bit (MSB), which is carried to the next higher bit position in multi-bit addition.

Boolean Expressions

- **Sum (S):** $S = A \oplus B \oplus C_{in}$
- **Carry-out (Cout):** $C_{out} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in})$

Truth Table of Full Adder

A	B	Cin	Sum(S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0

1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tool Required:

- Functional Simulation: Incisive Simulator (ncvlog, ncelab, ncsim).
- Synthesis: Genus

Creating a Workspace:

- In Desktop Create a folder and name it as Digital_Vlsi (give folder name without any space).
- Create a new sub-directory and name it as pipo_seq for the design and open a terminal from the sub-directory.

Functional Simulation:

- Invoke the cadence environment by type the below commands • csh (Invokes C-Shell)
- source /home/install/cshrc (mention the path of the tools) (The path of cshrc could vary depending on the installation destination as /home/install/or /home etc.)

Creating Source Codes:

- In the Terminal, type gedit .v
- A blank document opens up into which the following source code can be typed down.

Source Code

```
module fa(input a,b,cin,
output sum,cout);

assign sum = a ^ b ^ cin;
assign cout = (a&b)|(b&cin)|(a&cin);

endmodule
```

Creating Test Bench: Similarly, create your test bench using gedit <filename_tb>.v to open a new blank document.

```

module fa_tb();
reg a,b,cin;
wire sum, cout;

fa uut(.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

initial begin
$monitor("Time=%t, a=%b, b=%b, cin=%b, sum=%b, cout=%b", $time,a,b,cin,sum,cout);
a=0; b=0; cin=0; #10
a=0; b=0; cin=1; #10
a=0; b=1; cin=0; #10
a=0; b=1; cin=1; #10
a=1; b=0; cin=0; #10
a=1; b=0; cin=1; #10
a=1; b=1; cin=0; #10
a=1; b=1; cin=1; #10

$finish;
end
endmodule

```

Waveform Full Adder-



To Launch Simulation tool

- linux:/> nclaunch -new&
// “-new” option is used for invoking NCVERILOG for the first time for any design
- linux:/> nclaunch&
// On subsequent calls to NCVERILOG
- It will invoke the nclaunch window for functional simulation we can compile, elaborate and simulate it using Multiple.

To perform the function simulation, the following three steps are involved Compilation, Elaboration and Simulation.

Step 1: Compilation: Process to check the correct Verilog language syntax and usage

Inputs: Supplied are Verilog design and test bench codes

Outputs: Compiled database created in mapped library if successful, generates report else error reported in log file

Steps for compilation:

1. Create work/library directory (most of the latest simulation tools creates automatically).
2. Map the work to library created (most of the latest simulation tools creates automatically).
3. Run the compile command with compile options.

- Left side select the file and in Tools : launch verilog compiler with current selection will get enable. Click it to compile the code
- Worklib is the directory where all the compiled codes are stored while Snapshot will have output of elaboration which in turn goes for simulation

- After compilation it will come under worklib you can see in right side window.
- Select the test bench and compile it. It will come under worklib. Under Worklib you can see the module and test-bench.
- The cds.lib file is an ASCII text file. It defines which libraries are accessible and where they are located. It contains statements that map logical library names to their physical directory paths. For this Design, you will define a library called “worklib”.

Step 2: Elaboration: To check the port connections in hierarchical design

Inputs: Top level design/test bench Verilog codes

Outputs: Elaborate database updated in mapped library if successful, generates report else error reported in log file

Steps for elaboration – Run the elaboration command with elaborate options

1. It builds the module hierarchy.
2. Binds modules to module instances.
3. Computes parameter values.
4. Checks for hierarchical names conflicts.
5. It also establishes net connectivity and prepares all of this for simulation.

- After elaboration the file will come under snapshot. Select the test bench and elaborate it.

Step 3: Simulation: Simulate with the given test vectors over a period of time to observe the output behaviour.

Inputs: Compiled and Elaborated top level module name.

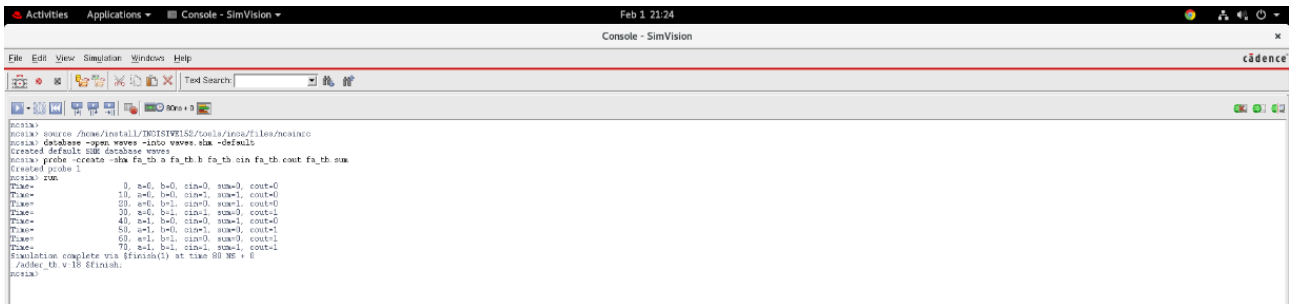
Outputs: Simulation log file, waveforms for debugging.

Simulation allow to dump design and test bench signals into a waveform.

Steps for simulation – Run the simulation command with simulator options.

RUN WITHOUT NCLaunch: Instead of NC Launch, design file and testbench can be run using single irun command. (Use your file name for .V)

```
[encmitpg@mit-ec-13 pipo_seq]$ irun pipo_seq.v pipo_tb.v -access +rwc -gui
```



Synthesize the design using Constraints and analyse reports, critical path and Max Operating Frequency

Step 1: Getting Started

- Make sure you close out all the Incisive tool windows first.
- Synthesis requires three files as follows,
 1. Liberty Files (.lib)
 2. Verilog/VHDL Files (.v or .vhdl or .vhd)
 3. SDC (Synopsis Design Constraint) File (.sdc)

Step 2 : Creating an SDC File

- In your terminal type “gedit counter_top.sdc” to create an SDC File if you do not have one.
- The SDC File must contain the following commands:

Constraint file:

- i. `create_clock -name clk -period 2 -waveform {0 1} [get_ports "clk"]`
- ii. `set_clock_transition -rise 0.1 [get_clocks "clk"]`
- iii. `set_clock_transition -fall 0.1 [get_clocks "clk"]`
- iv. `set_clock_uncertainty 0.01 [get_ports "clk"]`
- v. `set_input_delay -max 0.3 [get_ports "pi"] -clock [get_clocks "clk"]`
- vi. `set_output_delay -max 0.3 [get_ports "po"] -clock [get_clocks "clk"]`

i→ Creates a Clock named “clk” with Time Period 2ns and On Time from t=0 to t=1.

ii, iii $\rightarrow$ Sets Clock Rise and Fall time to 100ps.

- iv $\rightarrow$ Sets Clock Uncertainty to 10ps.

v, vi → Sets the maximum limit for I/O port delay to 1ps

Step 3 : Performing Synthesis

- The Liberty files are present in the below path,
/home/install/FOUNDRY/digital/<Technology_Node_number>nm/dig/lib/
- The Available technology nodes are 180nm ,90nm and 45nm.
- In the terminal, initialise the tools with the following commands if a new terminal is being used.
 - `ssh`
 - `source /home/install/cshrc`
- The tool used for Synthesis is “Genus”. Hence, type “`genus -gui`” to open the tool.
- The Following are commands to proceed :

1. `read_libs /home/install/FOUNDRY/digital/90nm/dig/lib/slow.lib`
2. `read_hdl seq.v`
3. `elaborate`
4. `read_sdc pipo.sdc` //reading top level sdc
5. `set_db syn_generic_effort medium` // effort level to medium for generic, mapping
6. `set_db syn_map_effort medium`
7. `set_db syn_opt_effort medium`
8. `syn_generic`
9. `syn_map`
10. `syn_opt` //Performing Synthesis Mapping and Optimisation
11. `report_timing > seq_timing.rep`
//Generates Timing report for worst datapath and dumps into file
12. `report_area > seq_area.rep`
//Generates Synthesis Area report and dumps into a file
13. `report_power > seq_pwr.rep`
//Generates Power Report [Pre-Layout]
14. `write_hdl>seq_net.v` //Creates readable Netlist File
15. `write_sdc > seq.sdc` //Creates Block Level SDC
16. `gui_show`

Genus Script file with .tcl file Extension

Make sure that the following file need to be present in the directory where the simulation is performed.

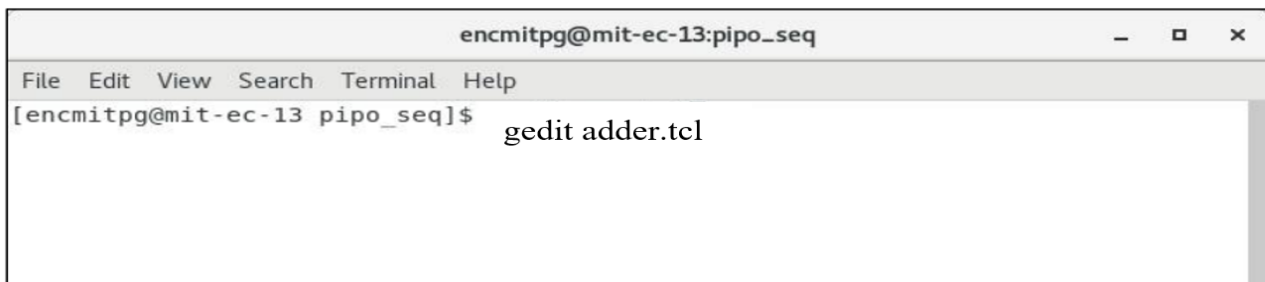
```
[150205105@aust synthesis_lab]$ tree
.
|-- EDI_files
|   |-- lef
|   |   |-- gsclib045.lef
|   |   |-- libs
|   |   |   |-- fast.lib
|   |   |   |-- slow.lib
|   |   |   |-- typical.lib
|   |   |-- others
|   |   |-- capTable
|-- input_files
|   |-- alu_4bit.v
|   |-- synthesis_cmd.tcl
5 directories, 7 files
```

Make sure the following commands are present inside the tcl (name.tcl) file

	Commands	Description
1	set_db init_lib_search_path EDI_files/libs/	Sets the value of a specific attribute. Here we are setting directory name where all the timing libraries are located.
2	set_db library slow.lib	Sets which timing library will be used while mapping
3	set_db lef_library EDI_files/lef/gsclib045.lef	Sets lef file of a target technology
4	set_db hdl_search_path input_files	Sets the directory name where RTL is located
5	read_hdl alu_4bit.v	Loads the design with pre-synthesized RTL
6	elaborate	Creates a design from Verilog module. Undefined modules are labeled as unresolved and treated as blackbox
7	set_top_module alu_4bit	Sets top module name
8	current_design alu_4bit	Changes the current directory in the design hierarchy to the specified design
9	write_hdl > alu_4bit_elaborated.v	Creates a structural netlist using generic/mapped logic
10	create_clock -name clk -period 10 [get_ports clk]	Creates a clock named "clk" having 10ns period in a specific port "clk"
11	set_clock_uncertainty -setup 0.5 [get_clocks clk]	Sets uncertainty value for the clocks while calculating setup
12	set_clock_uncertainty -hold 0.5 [get_clocks clk]	Sets uncertainty value for the clocks while calculating hold
13	set_max_transition 2 [get_ports clk]	Sets maximum allowable transition time for changing logic state to 2ns for data path
14	set_clock_transition -min -fall 0.5 [get_clocks clk]	Sets minimum allowable clock transition time to 0.5ns for switching logic state from high to low for clock path
15	set_clock_transition -min -rise 0.5 [get_clocks clk]	Sets minimum allowable clock transition time to 0.5ns for switching logic state from low to high for clock path
16	set_clock_transition -max -fall 0.5 [get_clocks clk]	Sets maximum allowable clock transition time to 0.5ns for switching logic state from high to low for clock path
17	set_clock_transition -max -rise 0.5 [get_clocks clk]	Sets maximum allowable clock transition time to 0.5ns for switching logic state from low to high for clock path
18	set_clock_groups -name original -group [list [get_clocks clk]]	Defines groups of specific clocks
19	set DRIVING_CELL BUFX8	Defines driving cell name which will drive the input ports of the design
20	set DRIVE_PIN {Y}	Defines driver pin of the driving cell
21	set_driving_cell -lib_cell \$DRIVING_CELL -pin \$DRIVE_PIN [all_inputs]	Sets driving cell properties for all the input ports
22	set_max_fanout 10 [current_design]	Sets maximum allowable fanout number to 10
23	set_load 0.5 [all_outputs]	Sets load capacitance of the output ports of the design
24	set_operating_conditions slow	Sets operating condition for delay calculation
25	set_input_delay -max 0.5 [all_inputs]	Synthesis tool assumes the data is launched by a positive edge triggered flop from the external logic

		(and the maximum input delay for the setup analysis is 0.5ns)
26	<code>set_output_delay -max 0.5 [all_outputs]</code>	Synthesis tool assumes the data is captured by a positive edge triggered flop in the external logic (and the maximum output delay for the setup analysis is 0.5ns)
27	<code>remove_assign -buffer_or_inverter BUFX16 -design [current_design]</code>	Removes assign statement using BUFX16 cell
28	<code>syn_generic</code>	Performs generic synthesis
29	<code>write_hdl > alu_4bit_generic.v</code>	Creates a structural netlist using generic logic after generic synthesis
30	<code>synthesize -to_mapped</code>	Performs mapping using target timing library
31	<code>write_hdl > alu_4bit_post_synthesis.v</code>	Creates a structural netlist using mapped logic after mapping
32	<code>remove_assigns_without_opt -buffer_or_inverter BUFX12 -verbose</code>	Removes assign statement using BUFX12 cell
33	<code>set_remove_assign_options -buffer_or_inverter BUFX12 -verbose</code>	Sets buffer or inverter cell to remove assign statements
34	<code>write -mapped > alu_4bit_mapped.v</code>	Writes mapped netlist for post-synthesis flow
35	<code>write_sdc > alu_4bit.sdc</code>	Writes constraints file for post-synthesis flow

Steps to create tcl file



```

encmitpg@mit-ec-13:pipo_seq
File Edit View Search Terminal Help
[encmitpg@mit-ec-13 pipo_seq]$ gedit adder.tcl

```

Fig. 53: Create tcl file

Write a tcl scripting file-

- If the circuits do not contain any clock signal then you can skip the .sdc file using the command “-unconstrained”.
- Further if you want to add delay information/ any input information for any circuits then .sdc file required


```
Activities Applications Text Editor Feb 1 20:48
Documents Open adder.tcl Save x
adder.tcl
read_libs /home/install/FUNDURY/digital/90nm/dig/lib/slow.lib
read_hdl adder.v
elaborate
read_sdc adder.sdc
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt
report_timing > adder_time.rep
report_area > adder_area.rep
report_power > adder_power.rep
write_hdl > adder_net.v
write_sdc > top_adder.sdc
gui_show

//without sdcfile hence adding -unconstrained in timing report
read_libs /home/install/FUNDURY/digital/90nm/dig/lib/slow.lib
read_hdl adder.v
elaborate
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt
report_timing -unconstrained > adder_time.rep
report_area > adder_area.rep
report_power > adder_power.rep
write_hdl > adder_net.v
gui_show
```

Run the created tcl file-

```
encmitpg@mit-ec-13:pipo_seq
File Edit View Search Terminal Help
[encmitpg@mit-ec-13 pipo_seq]$ gedit adder.tcl
[encmitpg@mit-ec-13 pipo_seq]$ genus -gui
```

```
Activities Applications Text Editor Feb 1 18:48
Documents Open adder_timing.rep Save x
adder_timing.rep
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1
Generated on: Feb 01 2025 06:44:43 pm
Module: fa
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed
Area mode: timing library

Path 1: UNCONSTRAINED
Startpoint: (F) b
Endpoint: (R) sum

Drv Adjust:+ 0 Launch 0
Data Path:- 304

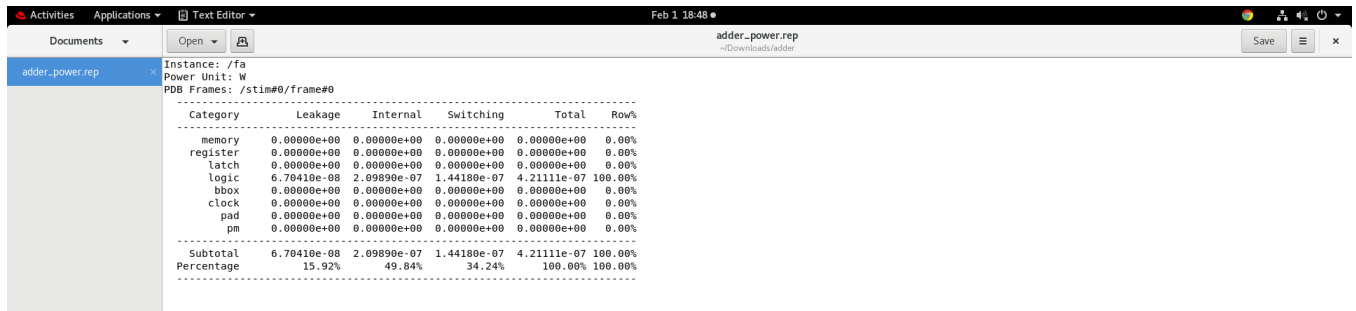
#-----
# Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival Instance
# (ff) (ps) (ps) (ps) Location
#-----
b - - F (arrival) 1 6.2 0 0 0 (-,.)
g8S_2398/S - B->S R ADDFXL 1 0.0 50 304 304 (-,.)
sum - - R (port) - - - 0 304 (-,.)
#-----
```

Area report-

```
Activities Applications Text Editor Feb 1 18:48
Documents Open adder_area.rep Save x
adder_area.rep
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1
Generated on: Feb 01 2025 06:43:55 pm
Module: fa
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed
Area mode: timing library

Instance Module Cell Count Cell Area Net Area Total Area Wireload
fa 1 19.679 0.000 19.679 <none> (0)
(D) = wireload is default in technology library
```

Power consumption report-



The screenshot shows a text editor window titled 'adder_power.rep' with the following content:

```
Instance: /fa
Power Unit: W
PDB Frames: /stim#0/frame#0
```

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	6.70410e-08	2.09890e-07	1.44180e-07	4.21111e-07	100.00%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	6.70410e-08	2.09890e-07	1.44180e-07	4.21111e-07	100.00%
Percentage	15.92%	49.84%	34.24%	100.00%	100.00%

Netlist files-

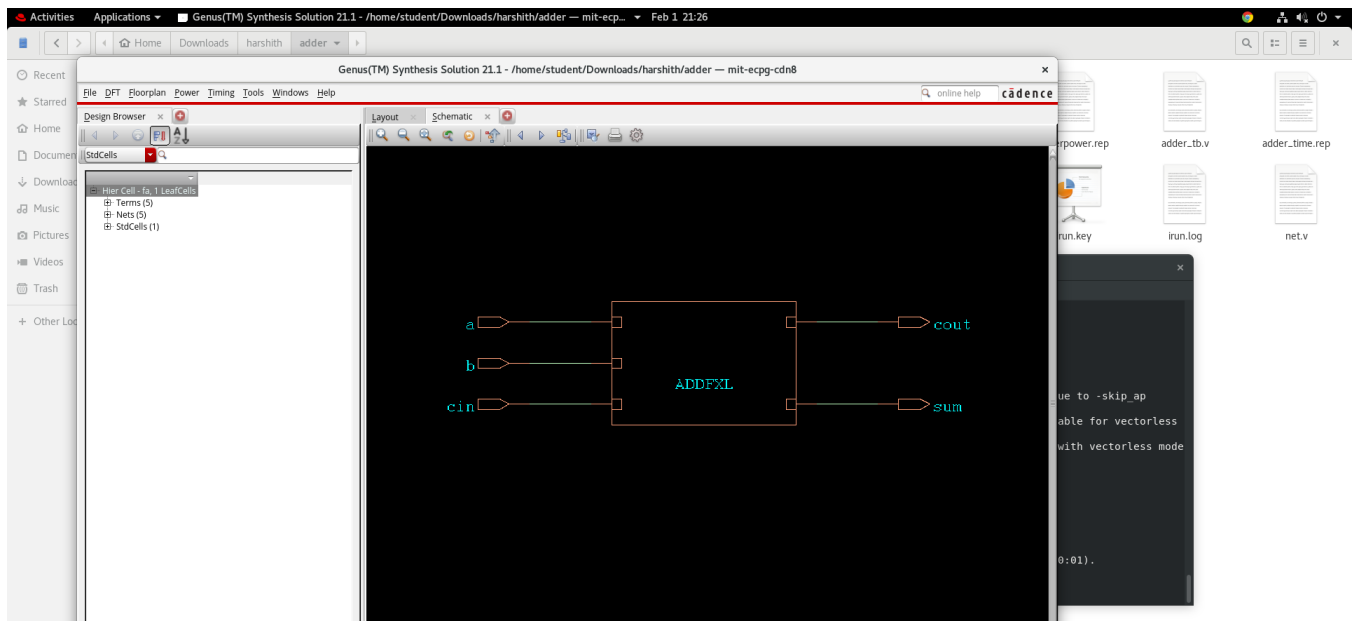


The screenshot shows a text editor window titled 'adder_netlist.v' with the following content:

```
// Generated by Cadence Genus(TM) Synthesis Solution 21.14-s082_1
// Generated on: Feb 1 2025 18:45:10 IST (Feb 1 2025 13:15:10 UTC)
// Verification Directory fv/fa

module fa(a, b, cin, sum, cout);
input a, b, cin;
output sum, cout;
wire a, b, cin;
wire sum, cout;
ADDFXL g85_2390(.A(a), .B(b), .CI(cin), .CO(cout), .S(sum));
endmodule
```

Schematic Diagram-



- To show the synthesized output execute the command `gui_show`
- The GUI window of the genus synthesis output will be opened. If you click and zoom into each block of the circuit, you will be able to view the gate interconnections inside the block.

Exercise problem:

1. Write a Verilog code for the following expression $Q = (AB)' \cdot (A+B)' \cdot C$ and do the synthesis. Further, calculate the power area and performance with and without using the constraint file.
2. Repeat the first problem by giving a delay to the input signal and checking the PPA.
3. s

Homework**• Exercise Problem- 1**

A Line follower Robot uses 2 Infra-Red (IR) sensors to detect the color of the surface. Sensor output is High when white surface is detected and Low on a black surface. Try to model the Robot controller and then implement its hardware? A Line follower Robot uses 2 Infra-Red (IR) sensors to detect the color of the surface. Sensor output is High when white surface is detected and Low on a black surface. Try to model the Robot controller and then implement its hardware?

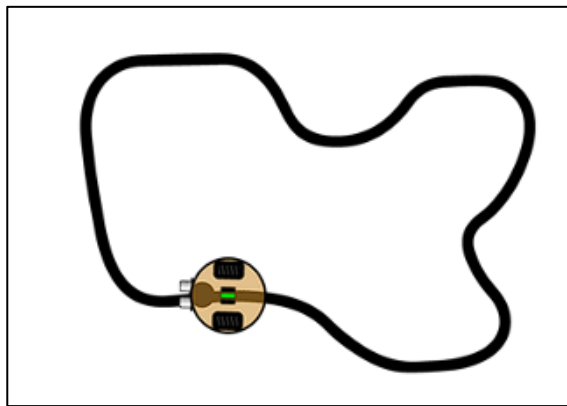


Figure 1: Line Follower

Source Code:

```
module LineFollowerRobot (  
    input clk, reset,  
    input sensorLeft, sensorRight, // Sensors detecting black or white surfaces  
    output reg motorLeft, motorRight  
);  
  
parameter STRAIGHT = 2'b00, TURN_LEFT = 2'b01, TURN_RIGHT = 2'b10, STOP = 2'b11;  
reg [1:0] state, next_state;  
  
always @(posedge clk or posedge reset) begin  
    if (reset)  
        state <= STRAIGHT;  
    else  
        state <= next_state;  
end  
  
always @(*) begin  
    case (state)
```

```

STRAIGHT: begin
    if (sensorLeft == 1 && sensorRight == 1)
        next_state <= STRAIGHT;
    else if (sensorLeft == 0 && sensorRight == 1)
        next_state <= TURN_LEFT;
    else if (sensorLeft == 1 && sensorRight == 0)
        next_state <= TURN_RIGHT;
    else
        next_state <= STOP;
end
TURN_LEFT: begin
    if (sensorLeft == 0 && sensorRight == 1)
        next_state <= TURN_LEFT;
    else if (sensorLeft == 1 && sensorRight == 0)
        next_state <= TURN_RIGHT;
    else
        next_state <= STOP;
end
TURN_RIGHT: begin
    if (sensorLeft == 1 && sensorRight == 0)
        next_state <= TURN_RIGHT;
    else if (sensorLeft == 0 && sensorRight == 1)
        next_state <= TURN_LEFT;
    else
        next_state <= STOP;
end
STOP: begin
    if (sensorLeft == 0 && sensorRight == 0)
        next_state <= STOP;
    else
        next_state <= STRAIGHT;
end
default: next_state <= STRAIGHT;
endcase
end
always @(*) begin
    case (state)
        STRAIGHT: begin
            motorLeft <= 1;
            motorRight <= 1;
        end
        TURN_LEFT: begin
            motorLeft <= 0;
            motorRight <= 1;
        end
        TURN_RIGHT: begin
            motorLeft <= 1;
            motorRight <= 0;
        end
    end
end

```

```

        STOP: begin
            motorLeft <= 0;
            motorRight <= 0;
        end
default: begin
    motorLeft <= 1;
    motorRight <= 1;
end
endcase
end
endmodule

```

Testbench:

```

module tb_LineFollowerRobot();
    reg clk, reset;
    reg sensorLeft, sensorRight;
    wire motorLeft, motorRight;

    LineFollowerRobot UUT (
        .clk(clk),
        .reset(reset),
        .sensorLeft(sensorLeft),
        .sensorRight(sensorRight),
        .motorLeft(motorLeft),
        .motorRight(motorRight)
    );

    always #5 clk = ~clk;

    initial begin
        clk=0;
        reset = 0;
        sensorLeft = 0;
        sensorRight = 0;

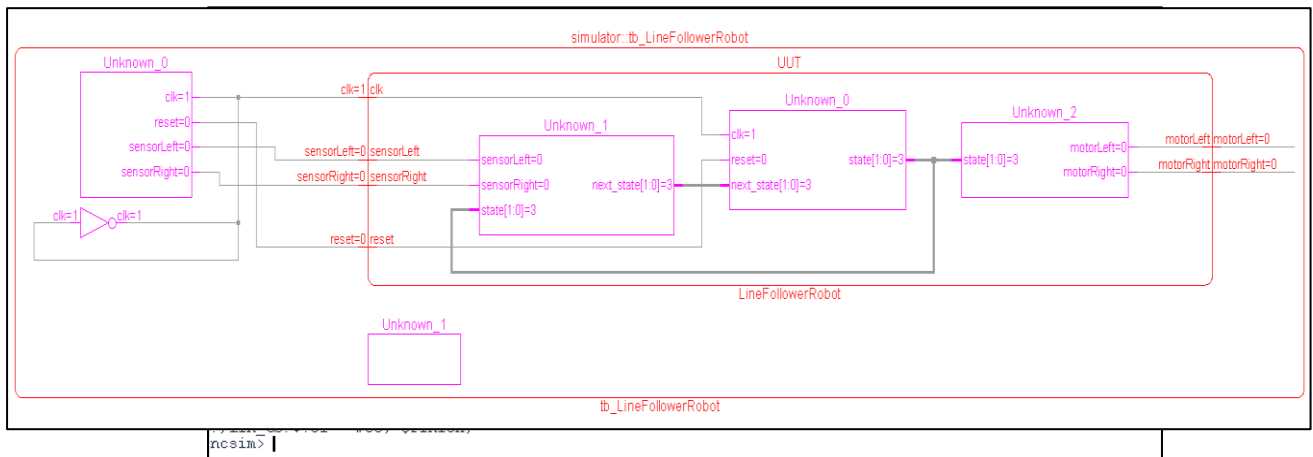
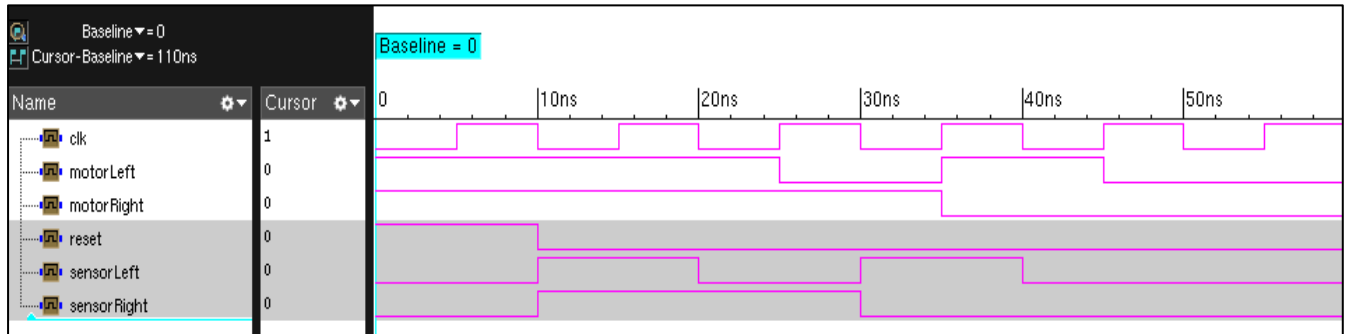
        reset = 1;#10;
        reset = 0;

        sensorLeft = 1; sensorRight = 1; #10;
        sensorLeft = 0; sensorRight = 1; #10;
        sensorLeft = 1; sensorRight = 0; #10;
        sensorLeft = 0; sensorRight = 0; #10;
        #60; $finish;
    end
    initial begin
        $monitor("Time %t: clk=%b, reset=%b, sensorLeft=%b,
        sensorRight=%b,motorLeft=%b,motorRight=%b",
        $time,clk,reset,sensorLeft,sensorRight,motorLeft,motorRight);
    end
endmodule

```

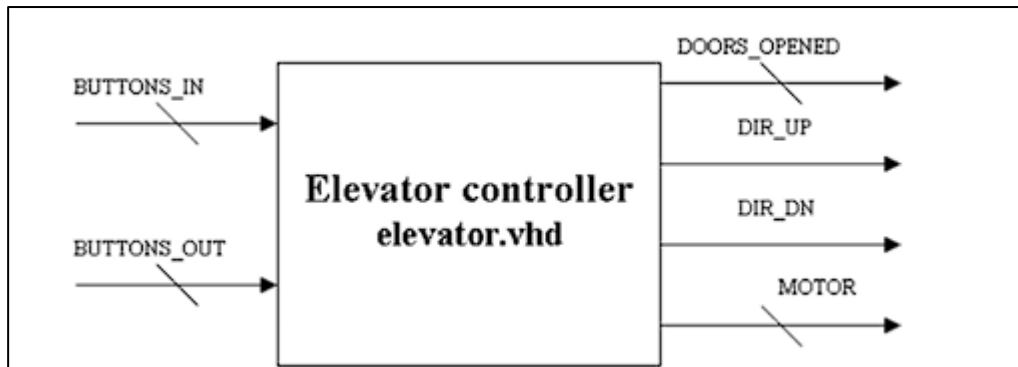
end
endmodule

Waveform and Simulation:



• Exercise Problem- 2

1. Model and implement an Elevator control system handling 4 floors (0 to 3)



Source Code:

```

module elevator(
    input clk,
    input reset,
    input [1:0] current_floor,
    input [3:0] buttons_in,
    input [3:0] buttons_out,
    output reg [3:0] motor,
    output reg dir_up,
    output reg dir_down
);

    reg [3:0] floor_requests;
    reg [1:0] next_floor;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            motor <= 4'b0000;
            dir_up <= 1'b0;
            dir_down <= 1'b0;
            next_floor <= current_floor;
            floor_requests <= 4'b0000;
        end
        else begin

            floor_requests <= buttons_in | buttons_out;

            if (floor_requests[current_floor]) begin
                motor <= 4'b0000; // Stop at current floor
                dir_up <= 1'b0;
                dir_down <= 1'b0;
            end
            else begin
                if (current_floor == 2'b00 && floor_requests[1]) begin
                    next_floor <= 2'b01;
                    motor <= 4'b0010;
                    dir_up <= 1'b1;
                    dir_down <= 1'b0;
                end
                else if (current_floor <= 2'b01 && floor_requests[2]) begin

```



```

        next_floor <= 2'b10;
        motor <= 4'b0100;
        dir_up <= 1'b1;
        dir_down <= 1'b0;
    end
    else if (current_floor <= 2'b10 && floor_requests[3]) begin
        next_floor <= 2'b11;
        motor <= 4'b1000;
        dir_up <= 1'b1;
        dir_down <= 1'b0;
    end

    else if (current_floor >= 2'b01 && floor_requests[0]) begin
        next_floor <= 2'b00;
        motor <= 4'b0001;
        dir_up <= 1'b0;
        dir_down <= 1'b1;
    end

    else if (current_floor >= 2'b10 && floor_requests[1]) begin
        next_floor <= 2'b01;
        motor <= 4'b0010;
        dir_up <= 1'b0;
        dir_down <= 1'b1;
    end

    else if (current_floor >= 2'b11 && floor_requests[2]) begin
        next_floor <= 2'b10;
        motor <= 4'b0100;
        dir_up <= 1'b0;
        dir_down <= 1'b1;
    end

    else begin
        motor <= 4'b0000; // No movement if no requests in any direction
        dir_up <= 1'b0;
        dir_down <= 1'b0;
    end
end
end
end
endmodule

```

Testbench:

```

module elevator_tb;

    reg clk;
    reg reset;
    reg [1:0] current_floor;
    reg [3:0] buttons_in;
    reg [3:0] buttons_out;
    wire [3:0] motor;
    wire dir_up;

```

```

wire dir_down;

elevator uut (
    .clk(clk),
    .reset(reset),
    .current_floor(current_floor),

.buttons_in(buttons_in),
    .buttons_out(buttons_out),
    .motor(motor),
    .dir_up(dir_up),
    .dir_down(dir_down)
);

always #5 clk = ~clk;

initial begin
    clk = 0;
    reset = 1;
    current_floor = 2'b00;
    buttons_in = 4'b0000;
    buttons_out = 4'b0000;

    #10;
    reset = 0;

    // Request to go to the 3rd floor (external)
    #10;
    buttons_out = 4'b1000; // Request for 3rd floor
    #20;
    current_floor = 2'b01; // Move to 1st floor
    #20;
    current_floor = 2'b10; // Move to 2nd floor
    #20;
    current_floor = 2'b11; // Reach 3rd floor
    #20;
    buttons_out = 4'b0000; // Clear requests

    // Request from 1st and 2nd floors (internal)
    #10;
    buttons_in = 4'b0110; // Requests for 1st and 2nd floors
    #20;
    current_floor = 2'b10; // Move to 2nd floor
    #20;
    buttons_in = 4'b0000; // Clear requests

    // Request to go down to ground floor (external)
    #10;
    buttons_out = 4'b0001; // Request for ground floor
    #20;

```

```

current_floor = 2'b10; // Move to 2nd floor
#20;
current_floor = 2'b01; // Move to 1st floor
#20;
current_floor = 2'b00; // Reach ground floor
#20;
buttons_out = 4'b0000; // Clear requests

```

```

$finish;
end

```

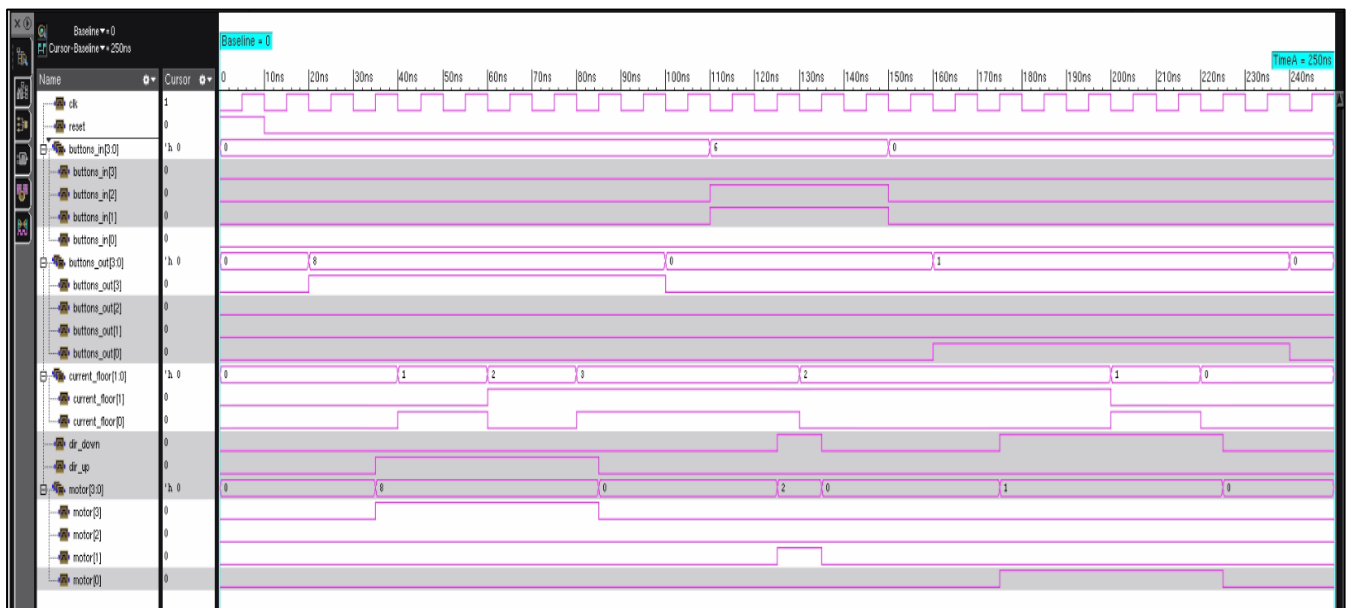
```

initial begin
    $monitor("Time=%0t | Floor=%0d | Buttons In=%b | Buttons Out=%b | Motor=%b | Dir_Up=%b | Dir_Down=%b",
        $time, current_floor, buttons_in, buttons_out, motor, dir_up, dir_down);
end

endmodule

```

Waveform and Simulation:



```

ncsim>
ncsim> source /home/install/INCISIVE152/tools/inca/files/ncsimrc
ncsim> database -open waves -into waves.shm -default
Created default SHM database waves
ncsim> probe -create -shm elevator_tb.buttons_in elevator_tb.buttons_out elevator_tb.clk eleva
Created probe 1
ncsim> run
Time=0 | Floor=0 | Buttons In=0000 | Buttons Out=0000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=20 | Floor=0 | Buttons In=0000 | Buttons Out=1000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=35 | Floor=0 | Buttons In=0000 | Buttons Out=1000 | Motor=1000 | Dir_Up=1 | Dir_Down=0
Time=40 | Floor=1 | Buttons In=0000 | Buttons Out=1000 | Motor=1000 | Dir_Up=1 | Dir_Down=0
Time=60 | Floor=2 | Buttons In=0000 | Buttons Out=1000 | Motor=1000 | Dir_Up=1 | Dir_Down=0
Time=80 | Floor=3 | Buttons In=0000 | Buttons Out=1000 | Motor=1000 | Dir_Up=1 | Dir_Down=0
Time=85 | Floor=3 | Buttons In=0000 | Buttons Out=1000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=100 | Floor=3 | Buttons In=0000 | Buttons Out=0000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=110 | Floor=3 | Buttons In=0110 | Buttons Out=0000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=125 | Floor=3 | Buttons In=0110 | Buttons Out=0000 | Motor=0010 | Dir_Up=0 | Dir_Down=1
Time=130 | Floor=2 | Buttons In=0110 | Buttons Out=0000 | Motor=0010 | Dir_Up=0 | Dir_Down=1
Time=135 | Floor=2 | Buttons In=0110 | Buttons Out=0000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=150 | Floor=2 | Buttons In=0000 | Buttons Out=0000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=160 | Floor=2 | Buttons In=0000 | Buttons Out=0001 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=175 | Floor=2 | Buttons In=0000 | Buttons Out=0001 | Motor=0001 | Dir_Up=0 | Dir_Down=1
Time=200 | Floor=1 | Buttons In=0000 | Buttons Out=0001 | Motor=0001 | Dir_Up=0 | Dir_Down=1
Time=220 | Floor=0 | Buttons In=0000 | Buttons Out=0001 | Motor=0001 | Dir_Up=0 | Dir_Down=1
Time=225 | Floor=0 | Buttons In=0000 | Buttons Out=0001 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Time=240 | Floor=0 | Buttons In=0000 | Buttons Out=0000 | Motor=0000 | Dir_Up=0 | Dir_Down=0
Simulation complete via $finish(1) at time 250 NS + 0
./elevator_tb.v:74 $finish;
ncsim> |

```

